



Project Report

Log Monitoring Dashboard

(Flask + React + MongoDB)

Group Members

Hamza Ali	2024203
Raja Hamza Sikandar	2024532
Hamza Mubasher	2024209

Public Deployment Link: <http://13.60.197.16>
GitHub Repository:
https://github.com/hamziAli/DevOps_Project.git

May 6, 2026

Contents

1	Introduction	3
1.1	Project Title	3
1.2	Project Objective	3
1.3	Overview	3
2	Technology Stack	3
2.1	Core Technologies	3
2.2	Why This Stack	4
3	Repository Architecture	4
3.1	Directory Structure	4
3.2	Architecture Description	4
4	Frontend Design and Implementation	5
4.1	Frontend Responsibilities	5
4.2	Frontend Containerization	5
4.3	Frontend Build Debugging	5
4.4	Frontend UI Screenshots	5
5	Backend Design and Implementation	6
5.1	Backend Responsibilities	6
5.2	Backend Testing	6
5.3	Backend Dockerization	6
6	Docker and Multi-Container Orchestration	6
6.1	Docker Compose Role	6
6.2	Benefits	7
7	CI/CD Pipeline with GitHub Actions	7
7.1	Workflow Configuration	7
7.2	Pipeline Stages	7
7.3	Workflow Screenshot	8
8	AWS Deployment and Automation	8
8.1	EC2 Launch Process	8
8.2	Secrets and Security	8
8.3	Automated Self-Healing Deployment Script	8
8.4	Deployment Screenshot	9

9	System Architecture Design	9
9.1	Architecture Explanation	9
9.2	Architecture Diagram	10
10	Task-Wise Implementation Summary	10
10.1	Task 1: Repository Architecture & Workflow Configuration	10
10.2	Task 2: Backend Testing Integration	10
10.3	Task 3: Frontend Containerization & Build Debugging	10
10.4	Task 4: Pipeline Automation & Infrastructure Self-Healing	11
11	Results and Validation	11
11.1	Key Outcomes	11
11.2	Public Access	11
12	Conclusion and Future Work	11
12.1	Conclusion	11
12.2	Future Improvements	12
	References	12

List of Figures

1	Frontend Dashboard - Main Interface	5
2	Frontend Dashboard - Log Monitoring View	6
3	GitHub Actions Workflow Execution	8
4	AWS EC2 Deployment and Service Startup	9
5	System Architecture of Log Monitoring Dashboard	10

1 Introduction

1.1 Project Title

This report presents the development and deployment of the **Log Monitoring Dashboard**, a full-stack DevOps project designed to collect, store, and display logs in real-time using a containerized architecture.

1.2 Project Objective

The key objective of this project is to implement a practical end-to-end DevOps pipeline that includes:

- Full-stack application development (frontend + backend),
- Containerization with Docker and Docker Compose,
- CI/CD automation through GitHub Actions,
- Cloud deployment on AWS EC2,
- Infrastructure setup through Terraform.

1.3 Overview

The solution follows a microservices-style architecture:

- **Frontend:** React (Vite) interface for users,
- **Backend:** Flask API for application logic and health checks,
- **Database:** MongoDB for persistent log storage.

All services are orchestrated through Docker Compose and deployed to AWS using an automated workflow.

2 Technology Stack

2.1 Core Technologies

Technology	Purpose
React (Vite)	Frontend single-page application
Flask (Python)	Backend REST API

MongoDB	Log data storage
Docker & Docker Compose	Containerization and multi-service orchestration
GitHub Actions	Continuous Integration and Continuous Deployment
AWS EC2	Cloud hosting and public accessibility
Terraform	Infrastructure as Code

2.2 Why This Stack

This stack was selected to demonstrate real-world DevOps practices with modular services, reproducible deployments, and automated testing/deployment workflows.

3 Repository Architecture

3.1 Directory Structure

```
Devops_Project/  
  backend/  
    app.py  
    requirements.txt  
    Dockerfile  
  frontend/  
    package.json  
    vite.config.js  
    src/  
  terraform/  
    main.tf  
  .github/workflows/  
    main.yml  
  docker-compose.yml  
  README.md
```

3.2 Architecture Description

The repository is organized into independent modules:

- **backend/** contains Flask API code and backend container build configuration.
- **frontend/** includes React source, Vite build setup, and production asset generation.
- **terraform/** defines cloud infrastructure resources.
- **.github/workflows/** contains CI/CD pipeline logic.

4 Frontend Design and Implementation

4.1 Frontend Responsibilities

The frontend provides:

- User interface for monitoring dashboard,
- API integration for backend communication,
- Production-ready static build served via Nginx.

4.2 Frontend Containerization

A multi-stage Docker build is used:

1. Build React assets in a Node.js stage.
2. Serve generated files from an Nginx Alpine image on port 80.

4.3 Frontend Build Debugging

During production build troubleshooting:

- Import paths were updated to include explicit `.jsx` extensions,
- Missing `App.jsx` was restored,
- Vite/Rollup Linux compatibility issues were resolved.

4.4 Frontend UI Screenshots

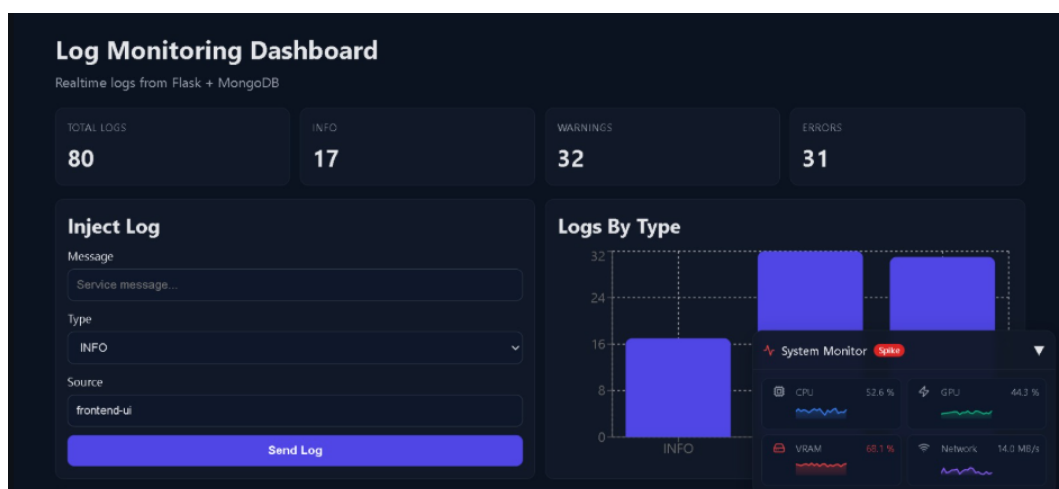


Figure 1: Frontend Dashboard - Main Interface

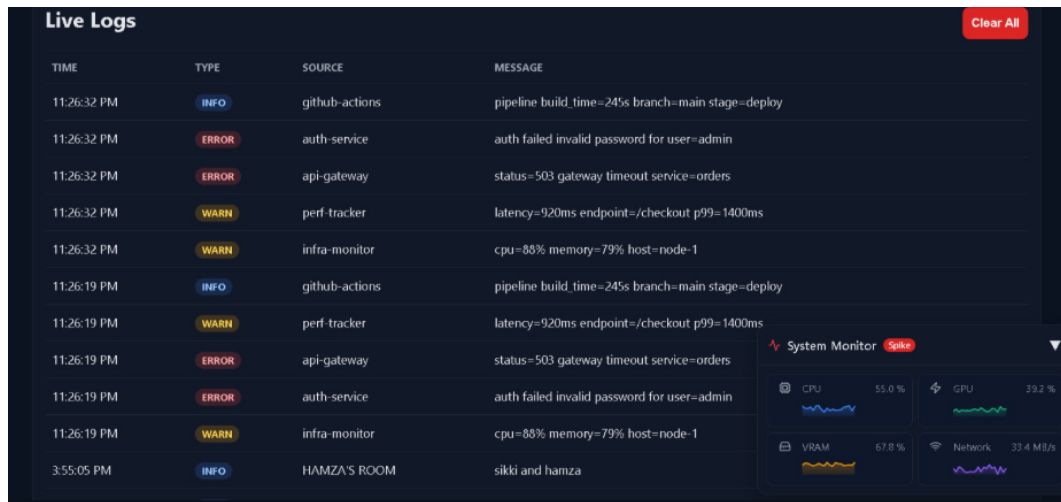


Figure 2: Frontend Dashboard - Log Monitoring View

5 Backend Design and Implementation

5.1 Backend Responsibilities

The Flask backend is responsible for:

- Exposing API endpoints (including `/health`),
- Handling server-side business logic,
- Integrating with MongoDB for data persistence.

5.2 Backend Testing

A dedicated `test_app.py` was implemented using `pytest` to validate backend stability and endpoint behavior. This enabled the CI test stage to pass successfully and provided confidence in automated deployments.

5.3 Backend Dockerization

The backend is containerized through a dedicated Dockerfile inside `backend/`. Required dependencies are installed from `requirements.txt`, and the Flask server is launched in a reproducible runtime environment.

6 Docker and Multi-Container Orchestration

6.1 Docker Compose Role

`docker-compose.yml` integrates all major services:

- Frontend container,
- Backend container,
- MongoDB database container.

6.2 Benefits

- One-command startup for complete system,
- Consistent local and cloud environments,
- Simplified dependency and networking management.

7 CI/CD Pipeline with GitHub Actions

7.1 Workflow Configuration

A correctly structured pipeline was configured under:

```
.github/workflows/main.yml
```

This ensured automatic trigger on repository pushes.

7.2 Pipeline Stages

1. Checkout source code
2. Setup runtime dependencies
3. Run backend tests (`pytest`)
4. Build and deploy application on AWS EC2

7.3 Workflow Screenshot

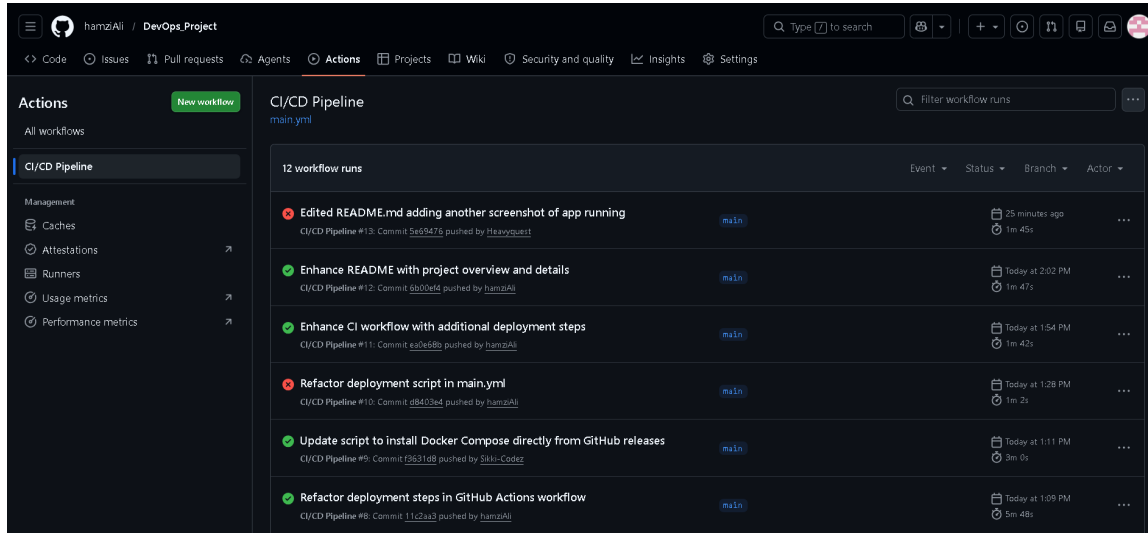


Figure 3: GitHub Actions Workflow Execution

8 AWS Deployment and Automation

8.1 EC2 Launch Process

The application was deployed on an Ubuntu 22.04 EC2 instance. Key setup steps included:

- Launching an EC2 instance from AWS Console,
- Enabling inbound rules for ports 22 (SSH) and 80 (HTTP),
- Using a generated `.pem` key for secure SSH access.

8.2 Secrets and Security

Sensitive data was stored in GitHub Secrets:

- `EC2_HOST` for server IP,
- `EC2_KEY` for private key content.

This protects credentials and avoids hardcoding confidential values in workflow files.

8.3 Automated Self-Healing Deployment Script

The deployment workflow was rewritten to:

- Install missing dependencies (git, Docker Compose plugin),

- Clone repository on remote server,
- Run `docker compose up -d` -build automatically.

This supports repeatable redeployments and minimizes manual intervention.

8.4 Deployment Screenshot

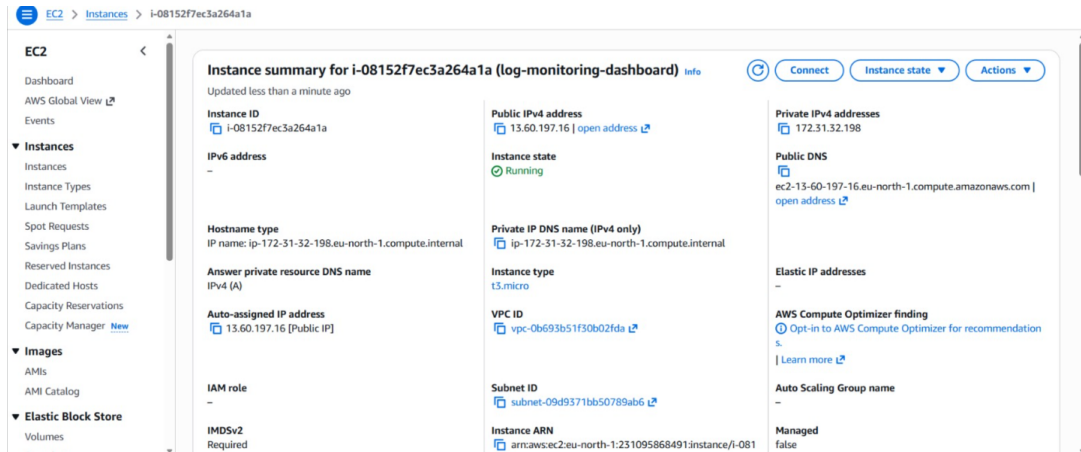


Figure 4: AWS EC2 Deployment and Service Startup

9 System Architecture Design

9.1 Architecture Explanation

The architecture follows a client-server model with containerized services:

- User accesses frontend via browser,
- Frontend communicates with Flask backend API,
- Backend reads/writes logs to MongoDB,
- Docker Compose handles service interconnections.

9.2 Architecture Diagram

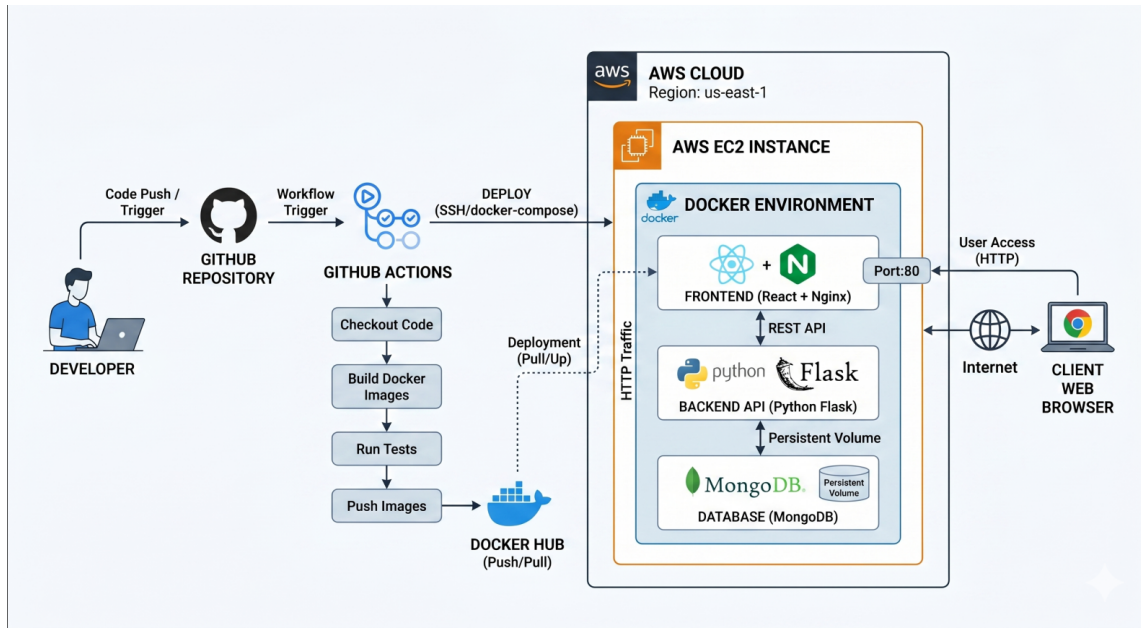


Figure 5: System Architecture of Log Monitoring Dashboard

10 Task-Wise Implementation Summary

10.1 Task 1: Repository Architecture & Workflow Configuration

- Created `.github/workflows` directory structure.
- Moved `main.yml` to correct location for auto-triggering.
- Reorganized root by introducing dedicated `backend/` directory.
- Migrated `app.py`, `requirements.txt`, and backend Dockerfile for proper build contexts.

10.2 Task 2: Backend Testing Integration

- Implemented `test_app.py` using `pytest`.
- Added endpoint validation for `/health`.
- Enabled successful CI test execution.

10.3 Task 3: Frontend Containerization & Build Debugging

- Authored multi-stage React Dockerfile.

- Used Node.js build stage and Nginx runtime stage.
- Fixed Linux-sensitive import path issues by appending `.jsx`.
- Restored missing `App.jsx` and stabilized production build.

10.4 Task 4: Pipeline Automation & Infrastructure Self-Healing

- Enhanced deployment script in `main.yml`.
- Automated dependency installation and Compose plugin setup.
- Added repository clone and container orchestration commands.
- Enabled end-to-end EC2 deployment via GitHub Actions.

11 Results and Validation

11.1 Key Outcomes

- Successfully deployed full-stack dashboard to public cloud.
- Automated testing and deployment pipeline established.
- Containerized architecture improved reproducibility.
- Modular repository structure improved maintainability.

11.2 Public Access

- Live URL: <http://13.60.197.16>
- Source Code: https://github.com/hamziAli/DevOps_Project.git

12 Conclusion and Future Work

12.1 Conclusion

This project demonstrates a practical, production-style DevOps workflow by combining software development, testing, containerization, CI/CD automation, and cloud deployment into one integrated system.

12.2 Future Improvements

- Add centralized log collectors and advanced filtering.
- Introduce role-based authentication for dashboard access.
- Add monitoring/alerting (Prometheus + Grafana).
- Expand Terraform modules for full infrastructure provisioning.

References

1. Project Repository: https://github.com/hamziAli/DevOps_Project.git
2. Public Deployment URL: <http://13.60.197.16>